



CMC UNIVERSITY

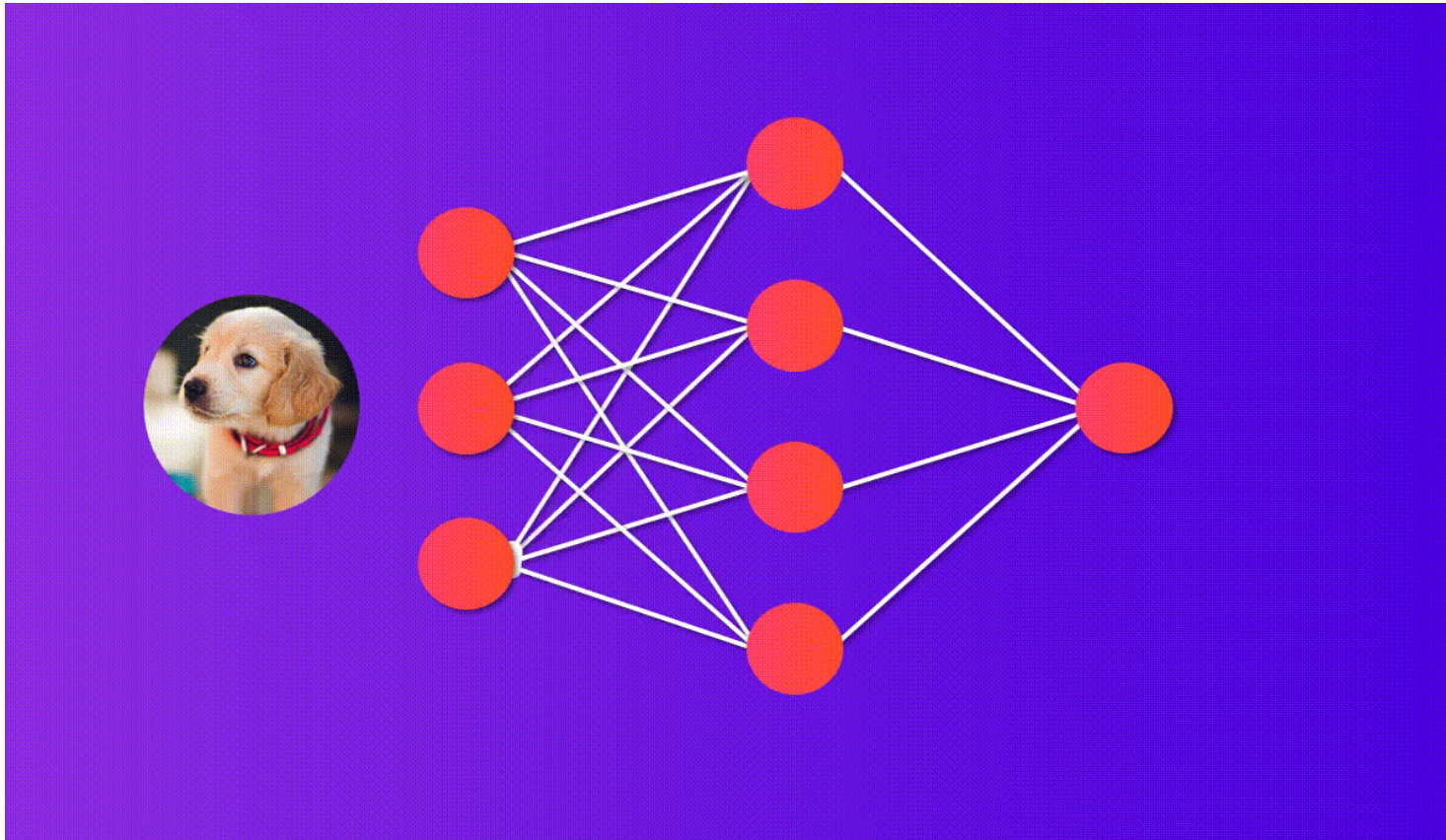
Aspire to Inspire the Digital World

Machine Learning and Data Mining

Ha Noi, 2026

LESSON 1.2

Neural Network



Neural networks are computational systems inspired by the structure and function of the human brain.

They consist of interconnected layers of artificial neurons that process and transmit information through weighted connections.

As neural networks grow deeper with more layers and neurons, they can learn increasingly complex functions, enabling deep learning models to excel in image recognition, natural language processing, and speech recognition.

Technology and Data

- Search engines
- Social media algorithms
- Recommendation systems

Input: User behavior
Output: Personalized content

Healthcare

- Medical image processing
- Disease diagnosis
- Personalized treatment planning

Input: X-ray/MRI images
Output: Pathology probability

Finance and Banking

- Fraud detection
- Credit risk assessment
- Portfolio management

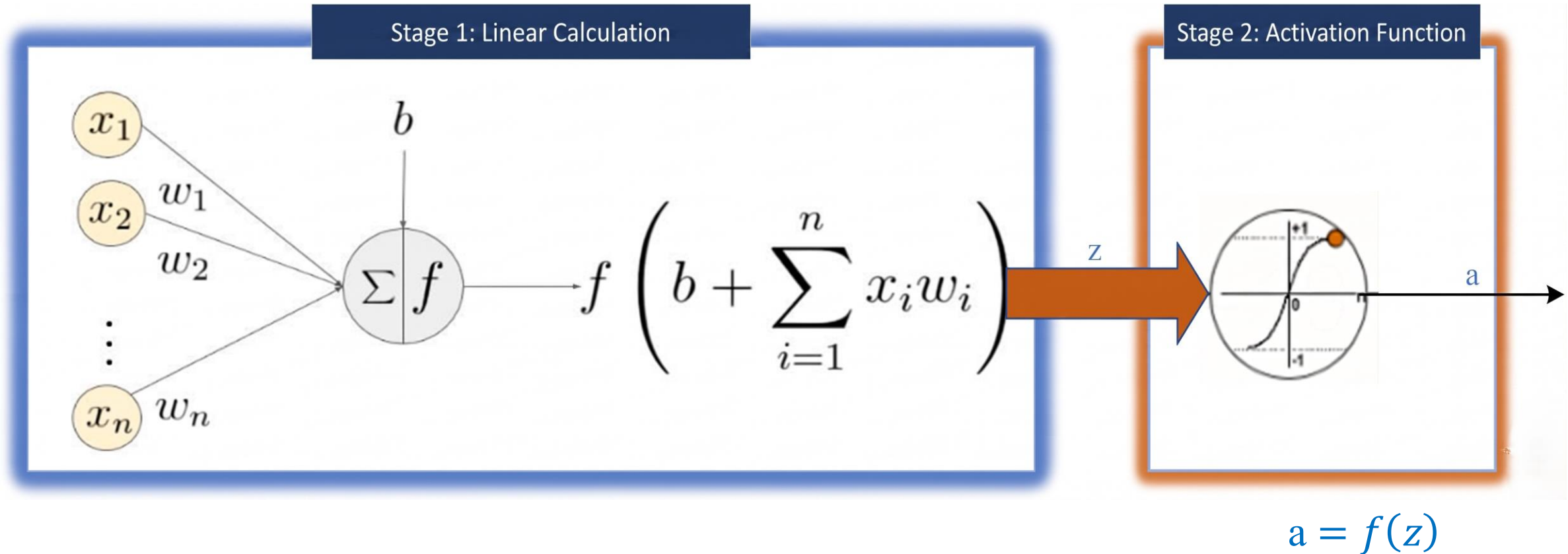
Input: Transaction history
Output: Risk/Safe

Commerce and Automation

- Supply chain optimization
- Customer segmentation
- Decision-making for self-driving cars

Input: Camera/Sensor data
Output: Driving trajectory

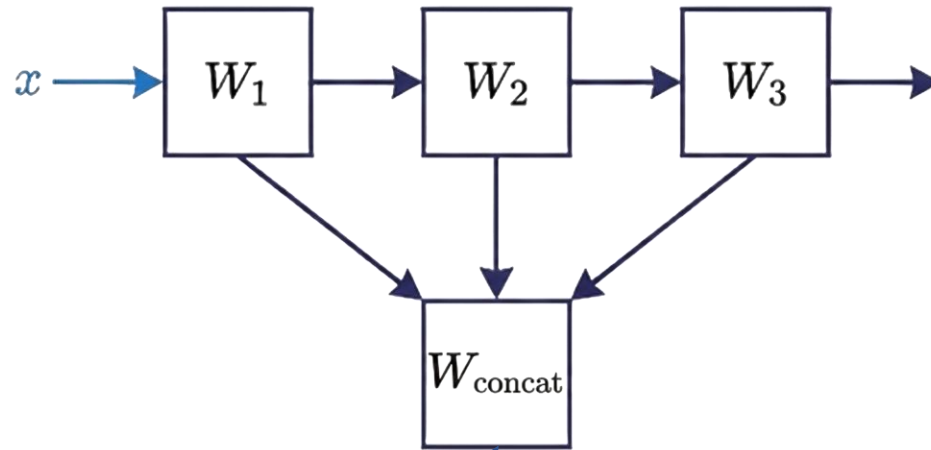
Why neural networks need activation functions



- $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$
- **Weight (w)**: Quantifies the importance of each input signal.
- **Bias (b)**: Shifts the activation threshold of the neuron.

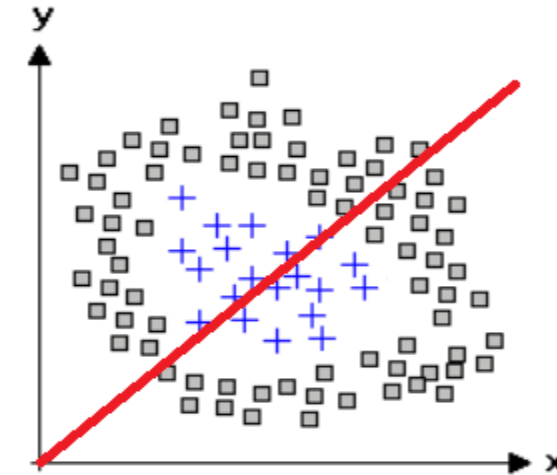
Decision Filter: Processes the z -value to decide which signals are 'activated', helping the network learn complex relationships.

The mathematical trap



$$f(x) = W_3(W_2(W_1x)) \rightarrow f(x) = W_{\text{concat}}x$$

Stacking an infinite number of linear layers still only creates a single linear boundary. The network loses its deep learning capability.



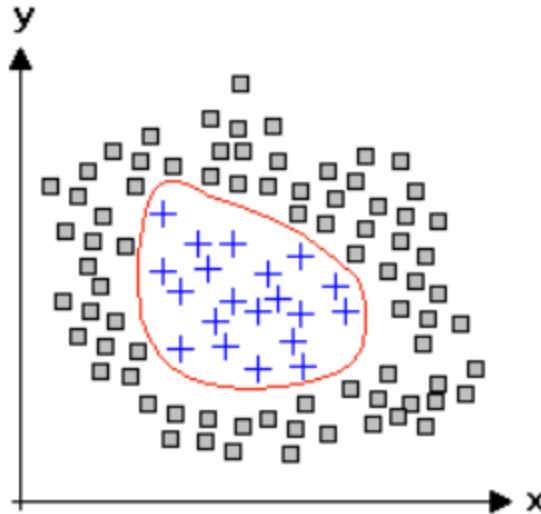
When do linear lines work well?

Simple problems (e.g., Classifying students as Pass/Fail based on scores).

Why do linear lines fail?

Real-world data is extremely complex (Face recognition, voice recognition). Cannot be accurately separated by a single linear cut.

The activation function is the "soul" that injects nonlinearity into the network, helping the model curve to match the complexity of the real world.



Bending Boundaries

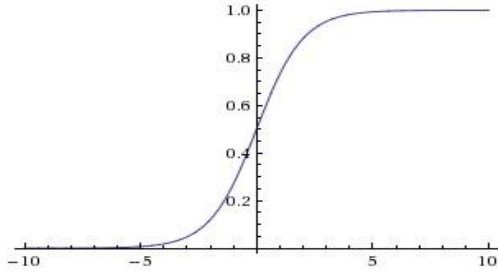
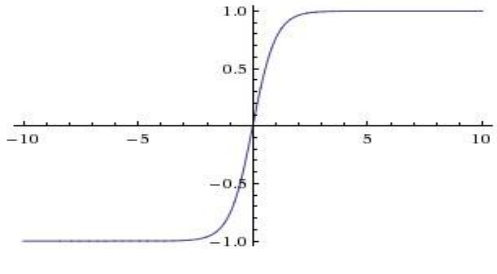
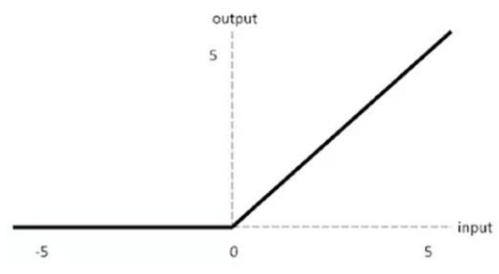
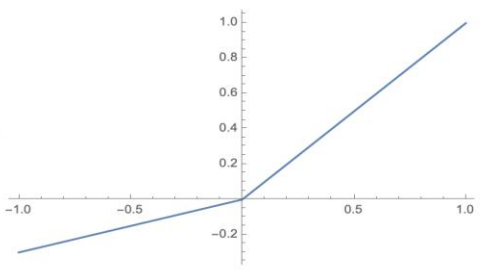
Bending Boundaries Extracting Complex Features The activation function introduces "nonlinearity" into the neural network. Instead of being locked into straight lines, the model can create multi-dimensional decision regions that closely follow real-world data patterns.

Complex Feature Extraction

Allows the network to learn deep, hidden nonlinear features. For example: In dog/cat recognition, the network can string together individual details (ears, eyes, shape, color) into a complete conclusion.

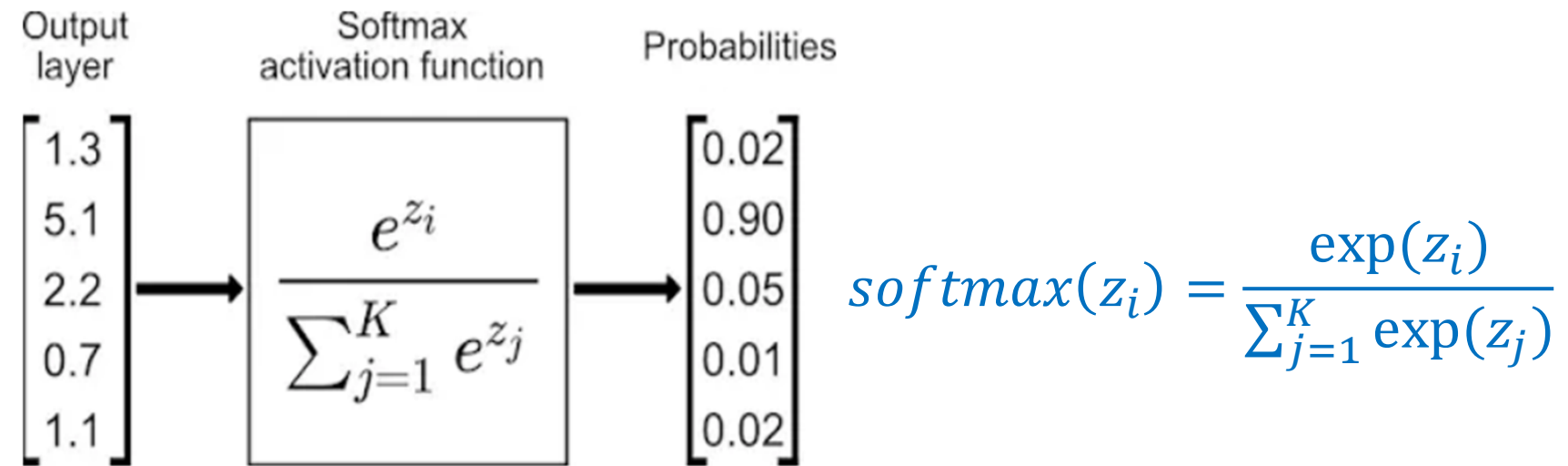
Activation functions are the ultimate key that transforms traditional Machine Learning into true Deep Learning.

Activation function

	Sigmoid	Tanh	ReLU	Leaky ReLU
Formula	 $\sigma(x) = \frac{1}{1 + e^{-x}}$	 $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	 $f(x) = \max(0, x)$	 $f(x) = \max(\alpha x, x)$
Output Range	(0, 1)	(-1, 1)	[0, ∞)	(-∞, ∞)
Advantages	Good for probability-based problems and binary classification.	Zero-centered, which helps the model converge faster than Sigmoid.	Extremely fast computation speed. Currently the most popular activation function.	Provides a small gradient when $x < 0$, effectively solving the "dying neuron" problem.
Disadvantages & Risks	High risk of vanishing gradients.	Still prone to vanishing gradients at the tails of the graph.	Prone to "dying neuron" issues when input values $x < 0$.	More complex due to the need for additional hyperparameter α tuning.

Function Softmax

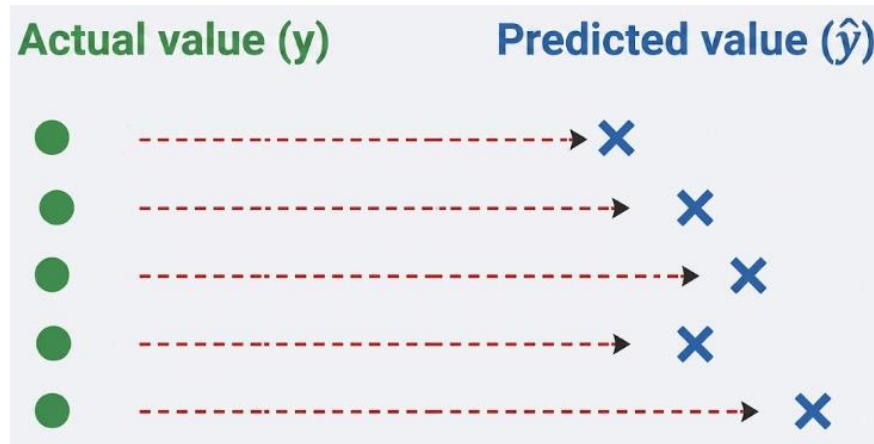
A specialized activation function placed at the output layer for classification problems with 3 or more classes.



Converts raw output values into a valid probability distribution. The sum of the output probabilities is always equal to 1.

Role:

- Tells the model how wrong it is.
- The smaller it is $\Rightarrow$ the better the model's prediction.
- It is the objective that the model needs to minimize during training.



Distance = Error

Two main types of problems

- Regression (Predicting continuous values $\Rightarrow$ use MSE)
- Classification (Predicting labels/probabilities $\Rightarrow$ use Cross-Entropy)

The goal of training is to find a set of weights w such that the Loss is minimized.

Applicable problem

- Regression: Predicting continuous values
- Examples:

House price prediction

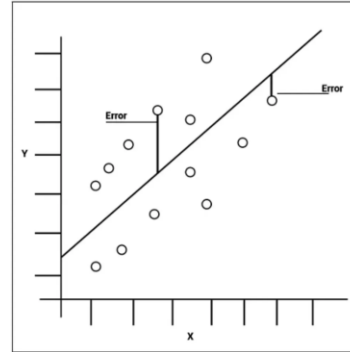
Temperature prediction

Revenue prediction

Formula:

$$L(w) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- y_i : Actual value
- $\hat{y}_i$: Model's predicted value
- N : Number of samples
- w : Model's weights



MSE heavily penalizes large errors due to the squaring mechanism.

Sample i	Actual y_i	Predicted $\hat{y}_i$	Error $y_i - \hat{y}_i$
1	3.0	2.5	0.25
2	5.0	4.0	1.00
3	2.0	2.5	0.25
4	7.24	4.0	10.5

Sum of squared errors = 12

Number of samples $N = 4$

$$\Rightarrow MSE = \frac{12}{4} = 3$$

The smaller the MSE $\Rightarrow$ the more accurate the prediction model.

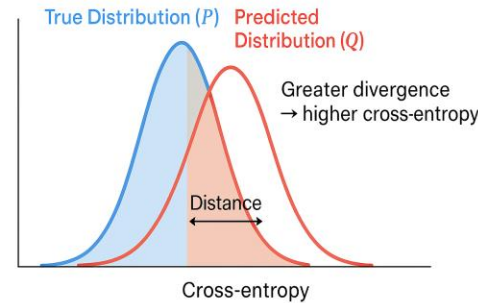
Applicable problem

- Regression: Predicting labels or probabilities
- Example:
Image recognition

Formula:

$$L(w) = - \sum_{i=1}^N y_i \cdot \log \hat{y}_i$$

- y_i : Actual value
- $\hat{y}_i$: Model's predicted value
- N : Number of samples
- w : Model's weights



**Larger distance $\Rightarrow$
Larger Cross-Entropy $\Rightarrow$ Stronger penalty**

3-class problem [Cat, Dog, Bird] with the actual class being “Dog”

Class	Actual	Predicted
Cat	0	0.95
Dog	1	0.03
Bird	0	0.02

If the model predicts correctly:

Example: Prediction [0.01, 0.98, 0.01] (actual is still "Dog")
 $L = -\log(0.98) = 0.0202$
 Almost correct $\Rightarrow$ Small Loss

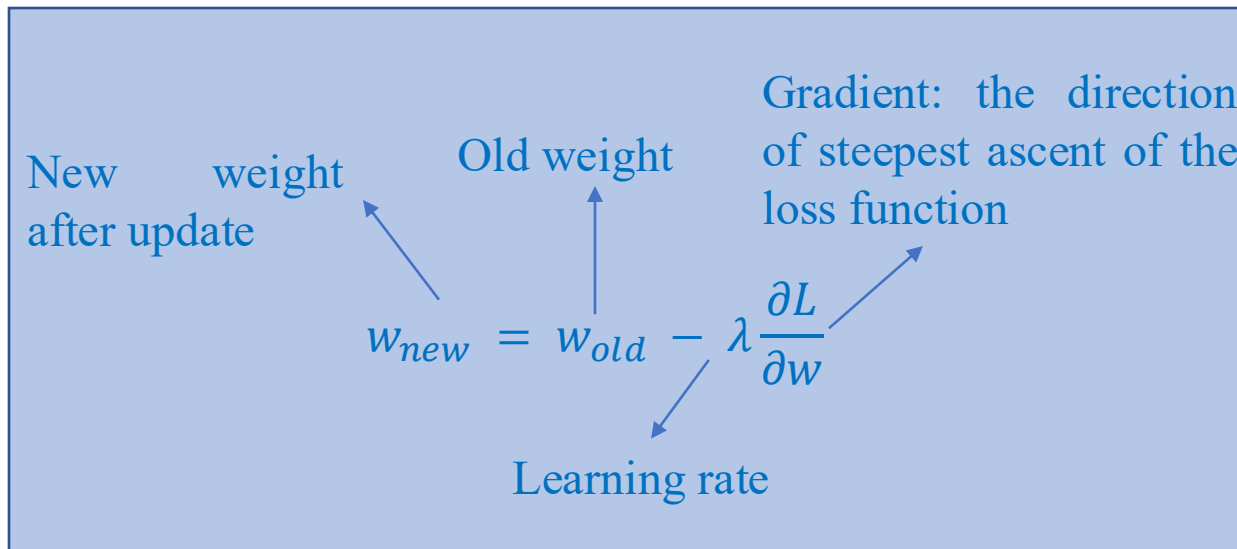
If the model predicts incorrectly but is overly confident:

Example: Prediction [0.95, 0.03, 0.02] (actual is "Dog" but the model is very confident incorrectly in class "Cat")
 $L = -\log(0.03) = 3.5066$
 Severely wrong $\Rightarrow$ Very large Loss

Gradient Descent is an algorithm that helps the model learn by gradually reducing the error (loss) to find the best set of weights.

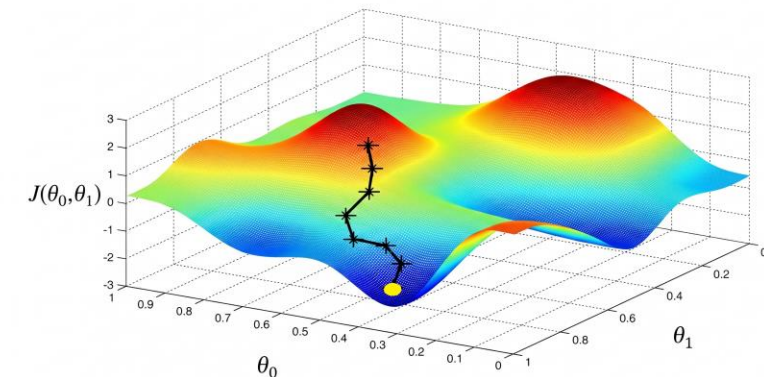
Objective:

- Find the set of weights w such that the loss function $L(w)$ is minimized.



$\frac{\partial L}{\partial w}$: indicates how the Loss L will change if w is changed slightly.

λ : determines the step size for each iteration.



✕ Weight update steps

● Optimal point (Minimum Loss)

Learning Rate (denoted as λ) determines the step size for each weight update.

Role of the Learning Rate: Helps balance learning speed and convergence capability

Learning Rate	Description	Result
λ is too large	Step size is too long	Prone to oscillation, difficult to converge or fails to converge.
λ is appropriate	Reasonable step size	Converges quickly and stably to the optimal point.
λ is too small	Step size is too short	Learns very slowly, consumes a lot of time and resources.

Tips for choosing a Learning Rate

- Start with a small value (e.g., 0.01, 0.001).
- If learning is slow $\Rightarrow$ gradually increase it.
- If the Loss oscillates $\Rightarrow$ gradually decrease it.
- You can use Learning Rate decay techniques during training.

Illustration of the impact of Learning Rate

Assume $J(\theta)$ is a loss function with respect to a parameter θ .

Objective: find θ^* such that $J(\theta)$ is minimized.



Characteristics:

- Very short steps.
- Takes a lot of time to reach the optimal point.

Consequence: Learns very slowly, wastes resources.

Characteristics:

- Reasonable steps.
- Descends steadily, quickly reaching the optimal point.

Consequence: Fast and stable convergence.

Characteristics:

- Steps are too long.
- Overshoots the optimal point, oscillates back and forth.

Consequence: Fails to converge or is very difficult to converge.

Gradient Descent finds the path downwards to reduce the Loss.
The Learning Rate determines our "step size" on that journey.
 $\Rightarrow$ Choosing the right Learning Rate = Fast learning + Good convergence!

Step 1 - Input Layer:

Raw data (x_1, x_2, x_3) is received into the network..



Step 2 - Hidden Layers:

Data passes through hidden layers. At each neuron, multiply the data by weights (W), add the bias, and pass it through a non-linear activation function.



Step 3 - Output:

Outputs the predicted result ($\hat{y}$). This result is compared with the actual value (y) determine the model's error.

Example box:

- x_1 : Attendance score
- x_2 : Assignment score
- x_3 : Midterm score

At each neuron:

Calculate the weighted sum:

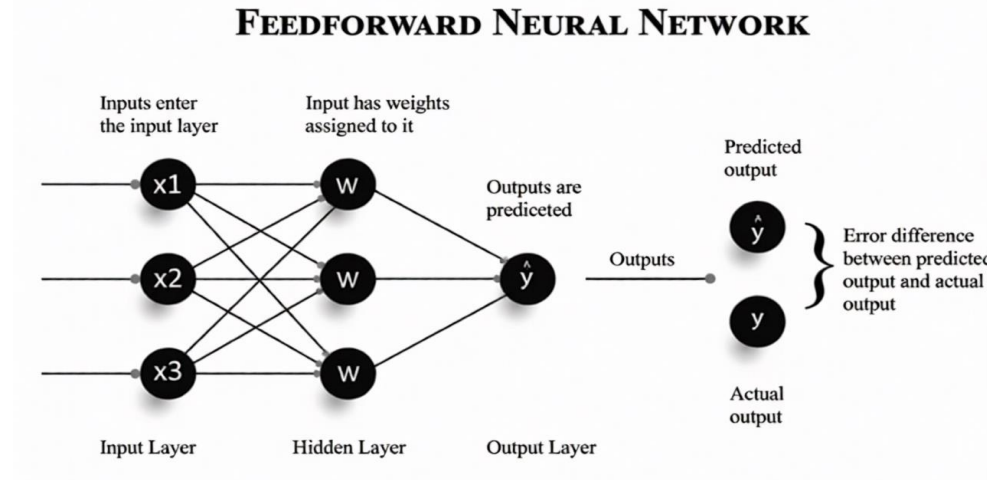
$$z = \sum_i x_i w_i + b$$

Apply activation function:

$$a = f(z)$$

Example:

- $\hat{y}$: Predicted final score
- y : Actual score
- Error = $\hat{y} - y$



Input Layer → Hidden Layer

Each hidden neuron receives all values from the input layer, multiplies them by weights (w), adds a bias (b), and then passes them through a non-linear activation function.

Hidden Layer → Hidden Layer (if there are multiple hidden layers)

The output from the previous hidden layer is passed to the next hidden layer to learn more complex features.

Hidden Layer → Output Layer

The final output from the hidden layer is passed to the output layer to generate the predicted result ($\hat{y}$).

With Feedforward, data flows in only one direction: from left to right.

Inputs:

- x_1 (Attendance) = 0.8
- x_2 (Assignment) = 0.6
- x_3 (Midterm) = 0.9

Input → Hidden

Hidden Neuron 1:

- Assume: $w = [0.5, 0.4, -0.1], b = 0.1$
- Calculate sum: $z_1 = 0.8(0.5) + 0.6(0.4) + 0.9(-0.1) + 0.1 = 0.65$
- Activation: $a_1 = \text{ReLU}(0.65) = 0.65$

Hidden → Output

Assume weights from hidden to output: $w = [1.2, -0.7], b = 0.05$

Calculate sum: $z_{out} = 1.2(0.65) + (-0.7)(0.22) + 0.05 = 0.676$

Predicted result (after ReLU activation): $\hat{y} = \text{ReLU}(0.676) = 0.676$

Comparison with actual value (y)

Assume: y (true score) = 0.70

Error: $y - \hat{y} = 0.70 - 0.676 = 0.024$

Neural Network:

- 3 inputs – 2 hidden neurons – 1 output
- Activation function: $\text{ReLU}(f(z) = \max(0, z))$

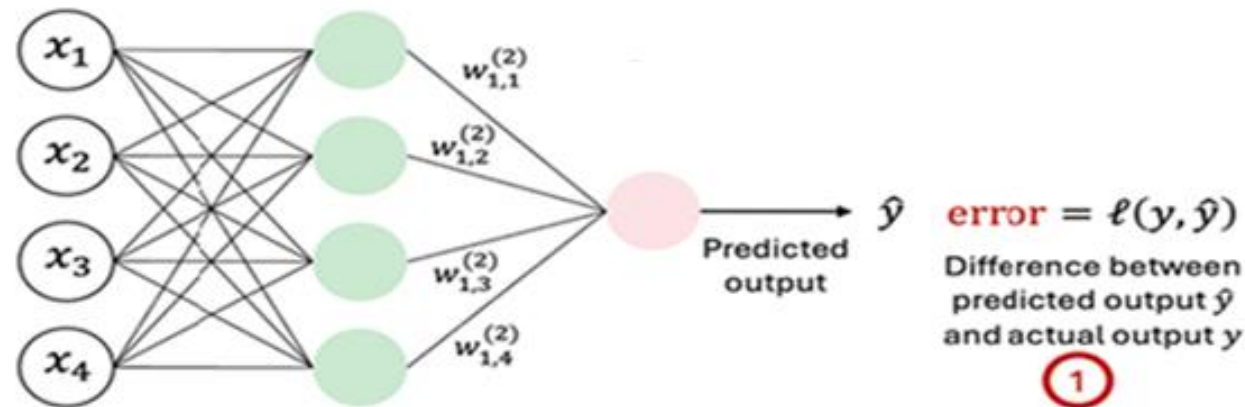
Hidden Neuron 2:

- Assume: $w = [-0.3, 0.2, 0.6], b = -0.2$
- Calculate sum: $z_2 = 0.8(-0.3) + 0.6(0.2) + 0.9(0.6) - 0.2 = 0.22$
- Activation: $a_2 = \text{ReLU}(0.22) = 0.22$

Step 1 - Error Calculation: Determining the discrepancy between predicted results and actual values.

Objective: The model should predict values as close to the actual values as possible.

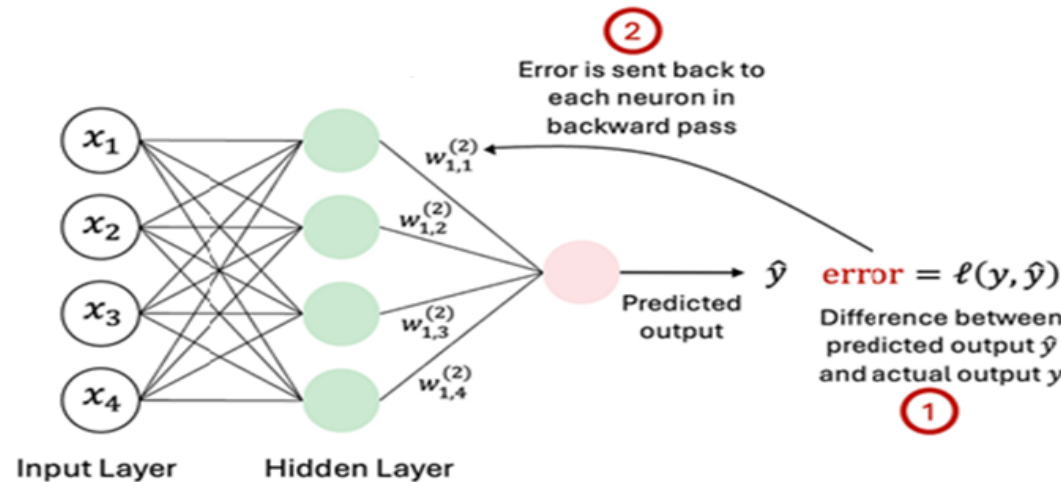
$$\text{error} = \ell(y, \hat{y})$$



Step 1 helps measure how much the model is currently "wrong." This error value serves as the starting point for adjusting the weights.

Step 2 - **Backward**: Send the error signal backward from the output layer to the neurons in the hidden layer.

Idea: Each neuron receives information on "how it contributed to the error" to know how it needs to change.



Error Propagation Flow

Start from the output: (from the predicted result $\hat{y}$ compared to the actual y).

Move backward to the hidden layer: (Hidden Layer).

Continue to neurons in previous layers: (all the way to the Input Layer with variables x_1, x_2, x_3, x_4).

What happens during backpropagation

- The error is "broken down" and sent back to each individual neuron.
- Each neuron calculates the portion of the error for which it is responsible.
- This signal continues to propagate backward until it reaches the input layer.

Notes

- The error is not just transmitted once!
- It is propagated through all connections (weights).

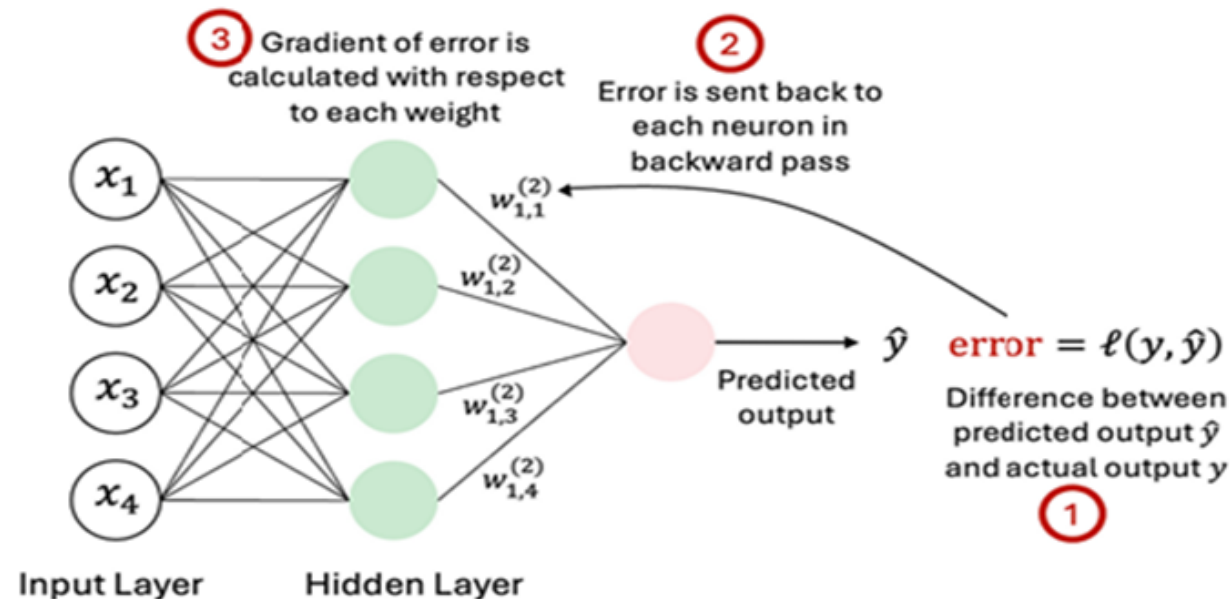
Step 3 - Gradient Calculation: Calculate the derivative of the error with respect to each weight to serve as a basis for weight adjustment, helping the model gradually reduce the error.

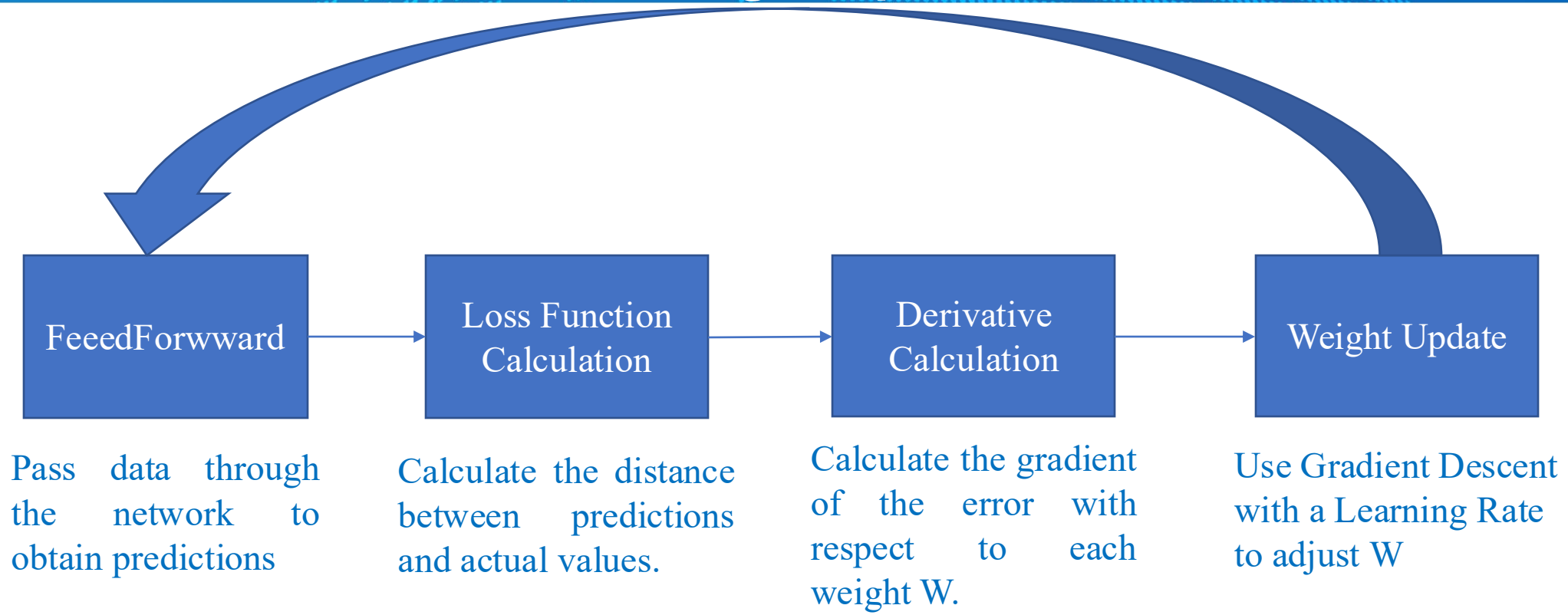
Idea: Calculate how the error would change if a weight is changed slightly.

Calculate Gradient
For each weight w , calculate the derivative: $\frac{\partial \ell(y, \hat{y})}{\partial w}$

Update Weights
Follow the direction that reduces the error:
$$w_{new} = w_{old} - \lambda \frac{\partial \ell(y, \hat{y})}{\partial w}$$

Repeat
Repeat the above process for all weights and train multiple times (multiple epochs) until the error is minimized.





TRAINING GOAL

Adjust the weights W so that the loss function becomes smaller over time, meaning the prediction $\hat{y}$ gets closer and closer to the actual value y . Repeat the above process for multiple epochs until the model converges..

Step 1: FeedForward

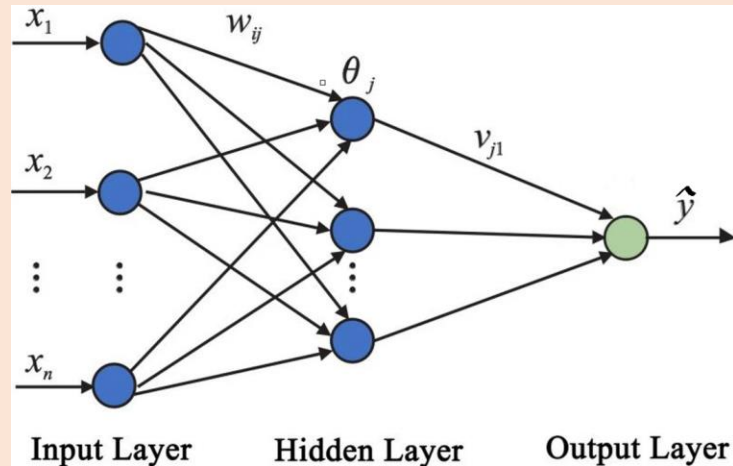


Diagram Components:

- **Input Layer:** Contains input variables $x_1, x_2, \dots, x_n$.
- **Hidden Layer:** Processes inputs using weights ($w_{\{ij\}}$) and biases.
- **Output Layer:** Produces the predicted output $\hat{y}$.

Calculation method

$$z = \sum_{i=1}^n w_i x_i + b$$

$$a = f(z)$$

Step 2: Loss function calculation

Actual
values:
 $(y_1), (y_2),$
 $\dots (y_n)$

Predicted
values: $\hat{y}_1,$
 $\hat{y}_2, \dots \hat{y}_n$

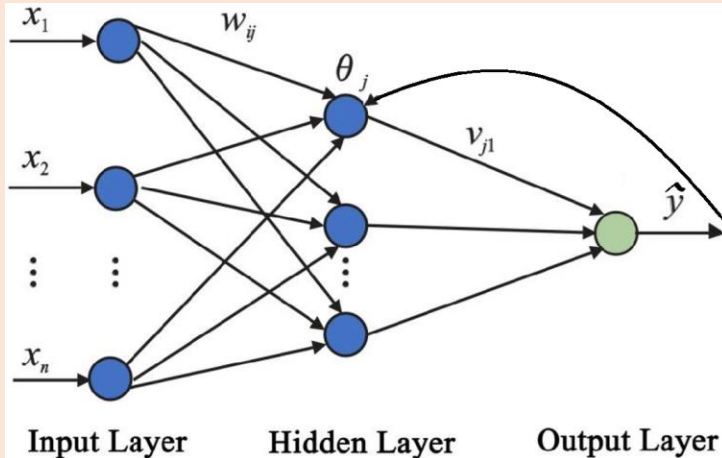
Measuring the model's error

$$L = \ell(y, \hat{y})$$

After step 2: We have an error value E that indicates how "wrong" the model is.

Next goal: Make E as small as possible.

Step 3: Calculate the derivative of the error with respect to weight W



Calculation method:

$$\frac{\partial L}{\partial w}$$

Significance:

- The error is "sent back" from output to input.
- Each weight receives a gradient value.

Step 4: Update weights using Gradient Descent

Adjust weights in the direction that reduces error with the formula:

$$w_{new} = w_{old} - \lambda \frac{\partial L}{\partial w}$$

Significance:

- Subtracting the gradient: moving in the opposite direction of error increase.
- Multiplying by the Learning Rate: controlling the step size.

After step 4, return to step 1 and repeat the entire process for multiple epochs until the error E is small enough or stops decreasing.